

POKEMON-Primary Type Classifier

Project Write-up

Live app: <https://pokemon-lc4qcddymp8wbeogrttsw4.streamlit.app/>

Repository: <https://github.com/sathyaamperayani/Pokemon>

Data source and why I chose it:

Data comes from Poke-API, a free REST API that requires no API key or registration a hard requirement for this project. It is well-documented, actively maintained, and offers a genuinely interesting classification problem: predicting a pokemon's primary type from its base stats. I chose it over the officially suggested alternatives (REST Countries, Open Brewery DB) because it required real pagination handling *and* a two-step fetch (list endpoint → per-item detail endpoint), giving more practice with the core skill the assignment targets, and because the resulting classification task is meaningfully harder than a country-region lookup, which made the model evaluation step more interesting to reason about.

Key cleaning decisions and why

Starting from 1,351 raw records, I made two decisions that mattered most:

1. **Removed 326 "alternate form" rows** (mega evolutions, gigantamax forms, regional variants identifiable by PokeAPI id > 10000). These represent the same species counted multiple times, which would bias the model toward species with many known variants. This was confirmed as a real issue, not a stylistic choice, by finding that all 34 "gmax" forms shared an identical weight value of exactly 10000 — a placeholder for missing data, not a real measurement.
2. **Left secondary type out of the model features entirely** (despite one-hot encoding it in the cleaned CSV for completeness). Using it to predict primary type would leak target information, since both describe the same underlying "typing" concept — a dashboard user wouldn't realistically know a Pokemon's secondary type without already knowing its type.

Missing base experience values (~3.6%) were median-imputed rather than dropped, to preserve otherwise-complete rows.

Model comparison results

Two classifiers were trained on 9 stat-based features (height, weight, base experience, hp, attack, defense, special attack, special defense, speed) to predict primary type (18 classes), using an 80/20 stratified split:

Model	Accuracy	Macro Precision	Macro Recall	Macro F1
Logistic Regression	0.215	0.133	0.148	0.135
Random Forest	0.215	0.140	0.152	0.138

Random Forest was selected for deployment — it matched or beat Logistic Regression on every metric, and its ability to capture non-linear combinations of stats (e.g. "high attack AND low defense AND high speed" as a joint signal) fits this problem better than a linear decision boundary.

Honest limitation

Both models land around 21.5% accuracy — roughly 4x better than the ~5.5% random-guess baseline for 18 classes, but far from strong in absolute terms. This isn't a bug to fix so much as a property of the problem: pokemon typing is a design/lore decision, not a deterministic function of base stats, so a Fire-type and a Fighting-type can have nearly identical stat profiles. A natural next step would be adding features the model currently can't see — e.g. move-pool composition, sprite color analysis, or generation/region — which likely carry more type-predictive signal than raw stats alone.